# Lists and indexing

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

---

TEACHER GUIDE · 1 OF 9

## Prepare the purpose, prerequisites and materials

> Prepare Lists and indexing: check the target, prerequisites, resources and adaptable teaching schedule.

### Teach the learning cycle

Primary target: Distinguish a **list index** from the value stored at that position

Session · Sequence and minutes · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.
>
> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min

> **Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Sections 9 and 11–14. Give extra time to finish the full task set; use Section 15 when the week includes a coding lab.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building.

Repeated copies of a case share one recorded attempt; they are not new **evidence**.

### Retrieval answer guidance

### Loops and trace tables

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

Look for: Trace variable values before and after each loop iteration

Reference example, not the only acceptable wording: 2, 3 and 4; the stop value is excluded. The body runs three times.

### Boolean conditions

Without opening the old lesson, explain one key idea from Boolean conditions. Give an example and a mistake that the idea helps you avoid.

Look for: Translate stated requirements into comparisons

Reference example, not the only acceptable wording: (minutes <= 12) and quiet. At most includes equality, so <= is required; the nonnegative integer and Boolean **input** contract still applies.

### From a paper algorithm to Python

Without opening the old lesson, explain one key idea from From a paper algorithm to Python. Give an example and a mistake that the idea helps you avoid.

Look for: Translate a short paper calculation into a Python script

Reference example, not the only acceptable wording: packs = 4; per_pack = 6; total = packs * per_pack; print(total), each on its own line. The first two bind quantities, the third computes 24 and the fourth displays 24.

Assignment alone produces no displayed **output** in this script.

Teach **list**s as ordered collections, with **index** and value kept distinct. The first exercises use nonnegative indices, although Python also permits negative **index**ing.

Reference: https://docs.python.org/3/tutorial/introduction.html#**list**s. Do not imply that updating a record changes or verifies the physical object represented.

### Outcomes

Distinguish a **list index** from the value stored at that position

Read and update an **item** using a valid **index**

Trace a loop through **list** values and test an empty **list**

### Preparation

Python 3 editor and **list**s.py

Four position cards numbered 0 through 3 and movable value cards

Blank trace table and separate predicted and observed **output** columns

### Prerequisite answer

0, 1 and 2; three iterations. The stop value 3 is excluded.

### End goal for the pictorial case

Trace **list**s and **index**ing and explain the changed program before running it.

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

# Explain and model the mechanism

> Model Lists and indexing using the supplied case, labelled mechanism and reasoning. Make the scope of each conclusion explicit.

A fictional supply shelf has three trays with counts 4, 7 and 4 in that order. Store them as counts = [4, 7, 4]. Square brackets enclose the **list** and commas separate its **item**s.

The order matters: the first and third trays contain equal values but occupy different positions.

## Explain Lists and indexing · 1

Python counts positions from zero. counts[0] reads the first **item**, counts[1] the second and counts[2] the third. len(counts) is 3 because there are three **item**s. It is a count, not the last nonnegative **index**.

This lesson uses integer counts and nonnegative indices; Python also supports negative indices, which are outside our current **index**ing convention.

## Explain Lists and indexing · 2

The printed lines are 3, 4, 7 and [4, 9, 4]. The assignment counts[1] = 9 replaces the **item** at **index** 1; it does not add a fourth **item** or change the two other values. The **length** stays 3.

A recorded count can be changed by code without proving that the physical tray really contains that many objects.

| Term | Meaning |
| --- | --- |
| List | An ordered collection of items |
| Index | An integer position used to access an item |
| Length | The number of items in a collection |
| Item | One value stored in a collection |
| IndexError | An error raised when an index is outside a sequence |

### Positions and stored counts

| TRAY POSITION | INDEX | VALUE |
| --- | --- | --- |
| First | 0 | 4 |
| Second | 1 | 7 |
| Third | 2 | 4 |

Equal stored values do not imply equal positions. There is no item at index 3.

## Explain Lists and indexing · 3

For [2, 0, 5], totals become 2, 2, 7. **Output** is 7. Zero is still an **item** and its iteration still executes. Exit: len([8, 3]) is 2, but **index** 2 is outside the **list**. Last nonnegative **index** is 1, whose value is 3.

An empty **list** has no last **item**, so a **length** check is required before access.

### Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a **prediction**, then explain the deciding step.

These teacher notes contain solutions; do not reveal the independent case answer during modelling.

```
counts = [4, 7, 4]
total = 0
for count in counts:
    total = total + count
print(total)
```

| Item read as count | total before | total after |
|---|---|---|
| 4 | 0 | 4 |
| 7 | 4 | 11 |
| 4 | 11 | 15 |

This fresh script prints 15. for count in counts supplies the stored values, not the indices. Both copies of 4 are **process**ed because both are **item**s. For an empty **list** written [], the body runs zero times and total stays 0.

For [7], it runs once and prints 7.

> **Change only the list to [2, 0, 5]. Trace the three additions and state the final output. Explain why the middle item is processed even though its value is zero.**

| Thinking move | Teacher prompt |
|---|---|
| Plan | What is given? What is the question? Which procedure fits these facts? |
| Do | Point to each input and intermediate step in the worked trace. Name the rule that allows the next step. |
| Check | Does the output answer the question? Which assumption or limitation prevents a larger claim? |

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

### A pictorial worked case: Lists and indexing

**Index**es are positions, not values. A **list** of shelf counts is read by position.

## The facts we will use

### Supplied facts

Python 3: counts=[6,1,8]; print(counts[0]); print(counts[-1]); print(len(counts)).




Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / PYTHON-BRIDGE/7

# Lists and indexing

**Start with the supplied facts**

Follow each arrow. Ask which fact allows the next step.



## 1. Draw positions

Indexes 0,1,2 contain 6,1,8 respectively.



## 2. Read two positions

Index 0 returns 6; index -1 returns the final value 8.



## 3. Check length boundary

Length is 3, but index 3 is beyond the final valid positive index 2.

**CHANGE ONE THING**

Replace the second print with print(counts[3]).

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

### Read the picture one step at a time

| STEP | WHAT HAPPENS AND WHY |
| --- | --- |
| 1. Draw positions | Indexes 0,1,2 contain 6,1,8 respectively. |
| 2. Read two positions | Index 0 returns 6; index -1 returns the final value 8. |
| 3. Check length boundary | Length is 3, but index 3 is beyond the final valid positive index 2. |

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

### Why this result follows

#### Why the result follows

**List** positions begin at zero. A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

### What this example does not establish

#### Keep the stated limit

A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.

**Change one thing in the pictured case**

### Change one thing

Change: Replace the second print with print(counts[3]).

### Predicted result

**Prediction**: The first print shows 6, then **IndexError** stops the script before the **length** print.

### Why the result follows

Reason: The attempted position does not exist.

**Ask while showing the picture**

### Ask while showing the picture · 4

Cover the result. Ask learners to name the **input**s and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its **evidence**. Then change the supplied condition and compare.

The picture is a representation; it does not document execution of a real AI model.

---



## Demonstrate and guide practice

> Demonstrate Lists and indexing, reduce support deliberately and check individual reasoning before discussing answers.

| Time or stage | Teacher action |
| --- | --- |
| Session 1 start to 7 minutes | Retrieve range(3) and its excluded stop. |
| 7 to 20 minutes | Place value cards under position cards and complete W1. |
| 20 to 33 minutes | Run the worked read and update script, then complete W2. |
| 33 to 45 minutes | Trace the value loop and answer the guided question. |
| Session 2 start to 8 minutes | Retrieve the distinction between length and final index. |
| 8 to 20 minutes | Predict the faulty access before running; use W3 to repair value versus index confusion. |
| 20 to 35 minutes | Collect W4 independently, keeping the replacement step separate from empty-list summation. |
| 35 to 45 minutes | Discuss W5 and use the exit to diagnose indexing assumptions. |

Use the worked example in Sections 3 and 6. Collect a **prediction** before revealing its result; ask the learner to point to each intermediate fact.

Keep position cards fixed while moving value cards. Permit an oral trace and provide a blank grid.

Extension: write a guarded final-**item** reader that prints Empty when len(readings) == 0, otherwise prints readings[len(readings) - 1]. Test [] and [6].

### Reduce help deliberately

#### Completion practice

Completion task: Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

### Reasoning to listen for

Teacher check: The result is [6, 7, 4]; **length** remains 3; **index** 2 still holds 4. Only the first **item** was replaced.

Start by identifying the given **input**s together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood.

For partners, alternate explainer and checker; collect a response from every learner before discussion.

#### Individual reasoning check

Use the worked case for **List**s and **index**ing. Proposed explanation: "**Length** is the last nonnegative **index**". Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

### Expected repair

Expected correction: **Length** counts **item**s; the last nonnegative **index** of a nonempty **list** is one less.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class.

If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

#### Pictorial practice question

Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

#### Reasoned teacher answer

Original **output**s are 6,8,3. After the change, 6 prints first, counts[3] raises **IndexError**, and the third print does not execute.

Changed-case reasoning: The first print shows 6, then **IndexError** stops the script before the **length** print. The attempted position does not exist.

Collect each learner's explanation before discussion. Use the first missing connection between **input**, deciding step and result to choose support. This question is worked-case practice, not fresh mastery **evidence**.

---

TEACHER GUIDE · 4 OF 9

## Run the investigation and diagnose misconceptions

> Run the Lists and indexing investigation and use the first unsupported step to choose a teaching response.

**Length** is the last nonnegative **index**: **Length** counts **item**s; the last nonnegative **index** of a nonempty **list** is one less.

A loop variable always contains an **index**: A direct **list** loop supplies stored values.

Updating one **item** grows the **list**: Assignment at an existing **index** replaces that **item**; **length** is unchanged.

Repeated values are **process**ed only once: Each position contributes one **item**, even when values repeat.

Use Section 9 for the **evidence** ledger. Add a boundary or missing-**input** case and record the expected response before checking. More successful repeats of one case do not establish wider coverage.

### Match the support to the first error

| What the response shows | Next teaching action |
| --- | --- |
| Missing or misread facts | Read the case aloud, mark supplied versus unknown facts, and let the learner restate it. |

| What the response shows | Next teaching action |
| --- | --- |
| Wrong procedure | Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation. |
| Arithmetic or implementation error | Trace intermediate values or lines; preserve the first mismatch and retry that step. |
| Correct result without a reason | Ask for a trace and a changed case before marking independent understanding. |
| An unsupported wider claim | Ask which supplied evidence supports the claim and which further observation would be needed. |

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal.

Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

---



## Assess and explain the answers

> Use the Lists and indexing questions, teacher keys and criteria after the relevant attempts. Previously discussed cases are practice; protect the separately selected independent task.

### Guided and exit questions

For [2, 0, 5], totals become 2, 2, 7. **Output** is 7. Zero is still an **item** and its iteration still executes. Exit: len([8, 3]) is 2, but **index** 2 is outside the **list**. Last nonnegative **index** is 1, whose value is 3.

An empty **list** has no last **item**, so a **length** check is required before access.

### W1 Separate index and value

#### Workbook question · W1

For counts = [4, 7, 4], give the **length**, valid nonnegative indices and value at **index** 1. Explain how the two **item**s with value 4 differ.

#### Teacher reasoning · W1

**Length** 3; indices 0, 1, 2; counts[1] is 7. The equal values occupy different positions, indices 0 and 2.

### W2 Trace an update

#### Workbook question · W2

Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

#### Teacher reasoning · W2

The result is [6, 7, 4]; **length** remains 3; **index** 2 still holds 4. Only the first **item** was replaced.

### W3 Repair a position mistake

#### Workbook question · W3

A learner loops with for count in counts and tries to read counts[count] for counts = [4, 7, 4]. Identify the first attempted **index** and error. Explain how to print each value directly.

### Teacher reasoning · W3

The first count is 4, so counts[4] raises **IndexError**. The loop variable already holds each value. Use an indented print(count) to print 4, 7 and 4 on separate lines.

## W4 Process new data independently

### Workbook question · W4

Start with readings = [3, 0, 3, 2]. Replace the second **item** with 5 using its **index**. Write a loop that adds all updated readings and prints one final total. Trace every update of the total.

Then test the summing part alone on [] and [6].

### Teacher reasoning · W4

Use readings[1] = 5, giving [3, 5, 3, 2]. Initialize total = 0 before for reading in readings, add reading inside, and print outside. Running totals are 3, 8, 11, 13; final 13. Summing [] gives 0 and [6] gives 6.

The replacement line itself requires at least two **item**s.

## W5 Keep the information you need

### Workbook question · W5

Compare [1, 5, 2] and [5, 1, 2]. State their totals and first **item**s. Would storing only the total answer which first **item** was larger? Explain.

### Teacher reasoning · W5

Both total 8. Their first **item**s are 1 and 5, so the second **list** has the larger first **item**. A total alone discards the order and cannot answer this question.

### Independent assessment and fresh retry

#### Teacher key · Independent assessment and fresh retry

W4 must use **index** 1 for the second **item**, retain all four **item**s, produce the full trace and **output** 13, and obtain 0 and 6 when testing only summation.

After support, retry [2, 2, 1], replace the first **item** with 4, and sum: updated **list** [4, 2, 1], totals 4, 6, 7. Then sum [] and [5] for **output**s 0 and 5. Record support before judging independence.

### Assess independent transfer

#### Independent question · withhold the key

Selected case: Predict the three **output**s. If the second print uses values[3] instead, where does execution stop and which later **output** is not produced?

```
values = [4, 9, 2]
print(values[1])
print(values[-1])
print(len(values))
```

### Selected case answer

#### Teacher key · Selected case answer

**Output**s are 9,2,3. With values[3], the first print produces 9, then **IndexError** occurs; the final **length** print is not reached. Positive valid indices are 0,1,2.

| Dimension | Evidence required |
| --- | --- |
| Explanation | Names the applicable idea or procedure and ties it to the supplied facts. |

| Dimension | Evidence required |
| --- | --- |
| Trace and result | Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution. |
| Limits | States the assumptions, missing evidence or conditions under which the result would change. |

### Teacher key · Selected case answer

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace.

Do not penalise an honestly recorded model failure if the learner correctly explains its **evidence** and limits.

### Feedback that the learner uses

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case.

Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

---

TEACHER GUIDE · 6 OF 9

## Connect the project, support and next lesson

> Connect Lists and indexing to Python readiness portfolio, adapt support and plan a later changed-case check.

Add a labelled **list**s and **index**ing **evidence** sheet to Python readiness portfolio. Show how you can distinguish a **list index** from the value stored at that position.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Keep position cards fixed while moving value cards. Permit an oral trace and provide a blank grid.

Extension: write a guarded final-**item** reader that prints Empty when len(readings) == 0, otherwise prints readings[len(readings) - 1]. Test [] and [6].

Week 8 groups reusable instructions in functions and supplies values through arguments.

Retain the project ledger and assess a changed case after support. Check the relevant prerequisite before carrying this component forward.

### Plan the delayed return

Suggested return points in this level: python-bridge/8, python-bridge/10. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used.

For a new level, revisit the previous capstone before checking the new prerequisites.

| Record | Teacher evidence |
| --- | --- |
| Learner code and date | |
| Concept and case used; prior exposure | |
| Explanation / trace / limits | |
| Support and first repair | |
| Changed case and delayed outcome | |
| Next teaching action | |

Use the **evidence** to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

### Project evidence from this case

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

---



## Prepare, teach and assess

> Use this preparation summary alongside the detailed Lists and indexing lesson; it is a checklist rather than a replacement for the full sequence.

### Teaching target

Teaching target: Distinguish a **list index** from the value stored at that position

Prepare the labelled worked-case picture, the matching workbook, fictional **input** cards and a place for individual responses. Check a local Python 3 environment before class; have a paper-trace version ready.

Session · Plan · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Plan:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min

> **Plan:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.
>
> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Plan:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Sections 9 and 11–14. Give extra time to finish the full task set; use Section 15 when the week includes a coding lab.

Before class, solve the supported practice: Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

### Supported practice answer

Reasoned practice answer: The result is [6, 7, 4]; **length** remains 3; **index** 2 still holds 4. Only the first **item** was replaced.

Check every learner before discussion: Use the worked case for **List**s and **index**ing. Proposed explanation: "**Length** is the last nonnegative **index**". Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

### Repair the misconception

Misconception repair: **Length** counts **item**s; the last nonnegative **index** of a nonempty **list** is one less.

### Support without giving away the answer

Support: read the exact case aloud, enlarge the labelled picture, supply **input** cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the

reasoning goal.

### Extend the reasoning

Extension: Replace the second print with print(counts[3]). Require a **prediction** and a reason before comparing. Expected result: The first print shows 6, then **IndexError** stops the script before the **length** print.

Reason: The attempted position does not exist.

### Protect the independent attempt

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

| Criterion | Not yet | With support | Independent |
|---|---|---|---|
| Explanation | Names an answer without a deciding fact | Uses a hint to link fact and decision | Explains the deciding fact in own words |
| Trace or test | Cannot connect input and result | Completes steps with prompts | Traces or tests a changed case accurately |
| Limits and revision | Claims more than evidence supports | Names a limit after a prompt | States a defensible limit and useful next test |

### Project checkpoint

Project checkpoint: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.

Use the weekly teacher answer for topic-specific reasoning. A saved progress tick is an activity record; judge mastery from the learner's explanation and **evidence**.

TEACHER GUIDE · 8 OF 9

## Use feedback, labs and repair paths

> Use feedback, earlier review and portfolios purposefully. Apply the coding routine only when this lesson has an executable lab.

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding **evidence** rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

Ask students to predict before running code. Preserve errors and actual **output**s; collect a changed test and an explanation. The branch-boundary and **index**ing lessons deliberately include failure cases.

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

Collect the completed-work portfolio as **evidence**. Record support separately from correctness, and assess explanation, trace/test and limitations. Progress ticks and quiz selections alone do not establish mastery.

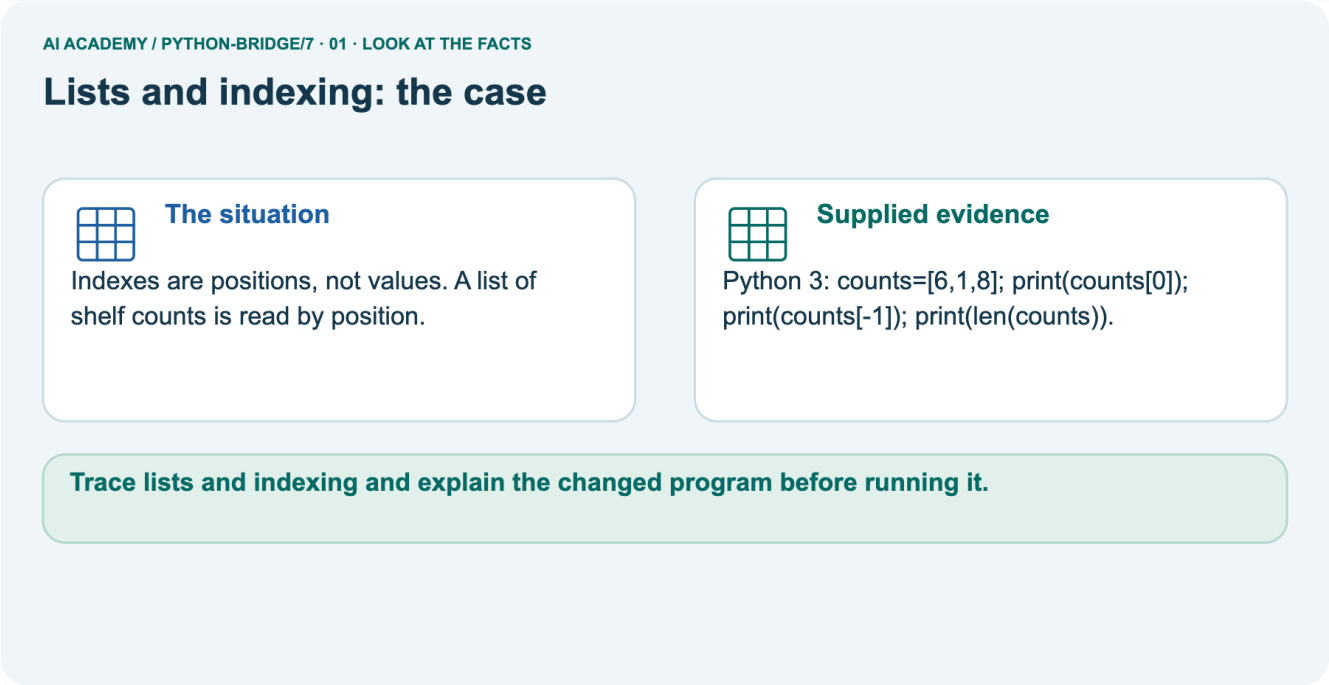
TEACHER GUIDE · 9 OF 9

## Visual concept journey

> Show one Lists and indexing graphic at a time, request a prediction and discuss its trace before the independent assessment.

Use one picture at a time. Hide a worked result before asking for a **prediction**, then reveal it for feedback. These are supported teaching cases; retain the separate independent assessment for a fresh first attempt.

### 01. Lists and indexing: the case




Follow the labelled picture with the matching explanation. Open the vector diagram

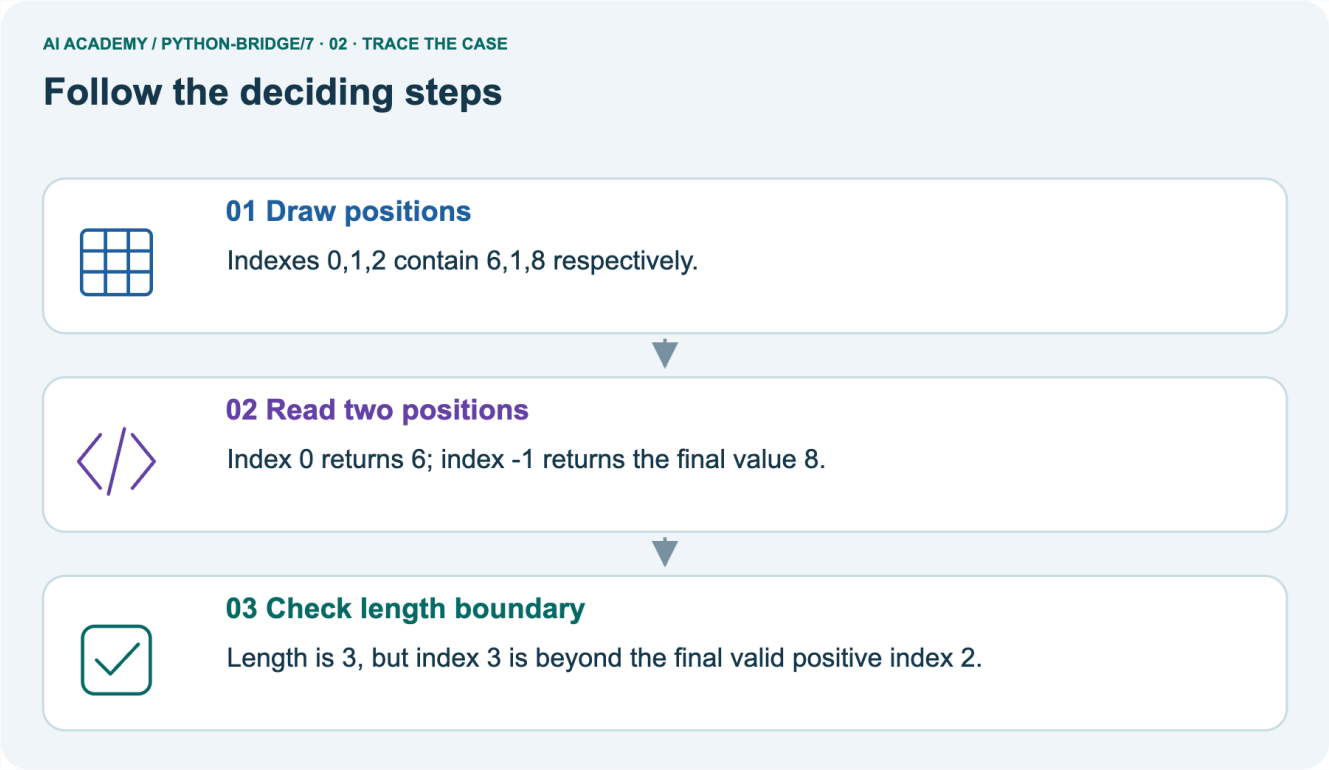
**Index**es are positions, not values. A **list** of shelf counts is read by position. Supplied facts: Python 3: counts=[6,1,8]; print(counts[0]); print(counts[-1]); print(len(counts)).

Point to the given fact that matters most. Explain why it matters.

#### Discuss while showing the picture

Teacher prompt: Point to the given fact that matters most. Explain why it matters. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

### 02. Follow the deciding steps




Follow the labelled picture with the matching explanation. Open the vector diagram

Draw positions: **Index**es 0,1,2 contain 6,1,8 respectively. → Read two positions: **Index** 0 returns 6; **index** -1 returns the final value 8. → Check **length** boundary: **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2.

Point to each numbered stage and explain which recorded **input** it uses.

#### Discuss while showing the picture

Teacher prompt: Point to each numbered stage and explain which recorded **input** it uses. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

### 03. Zero-based list positions




Follow the labelled picture with the matching explanation. Open the vector diagram

Zero-based **list** positions. Stored counts: **Index** 0 = 6, **Index** 1 = 1, **Index** 2 = 8.

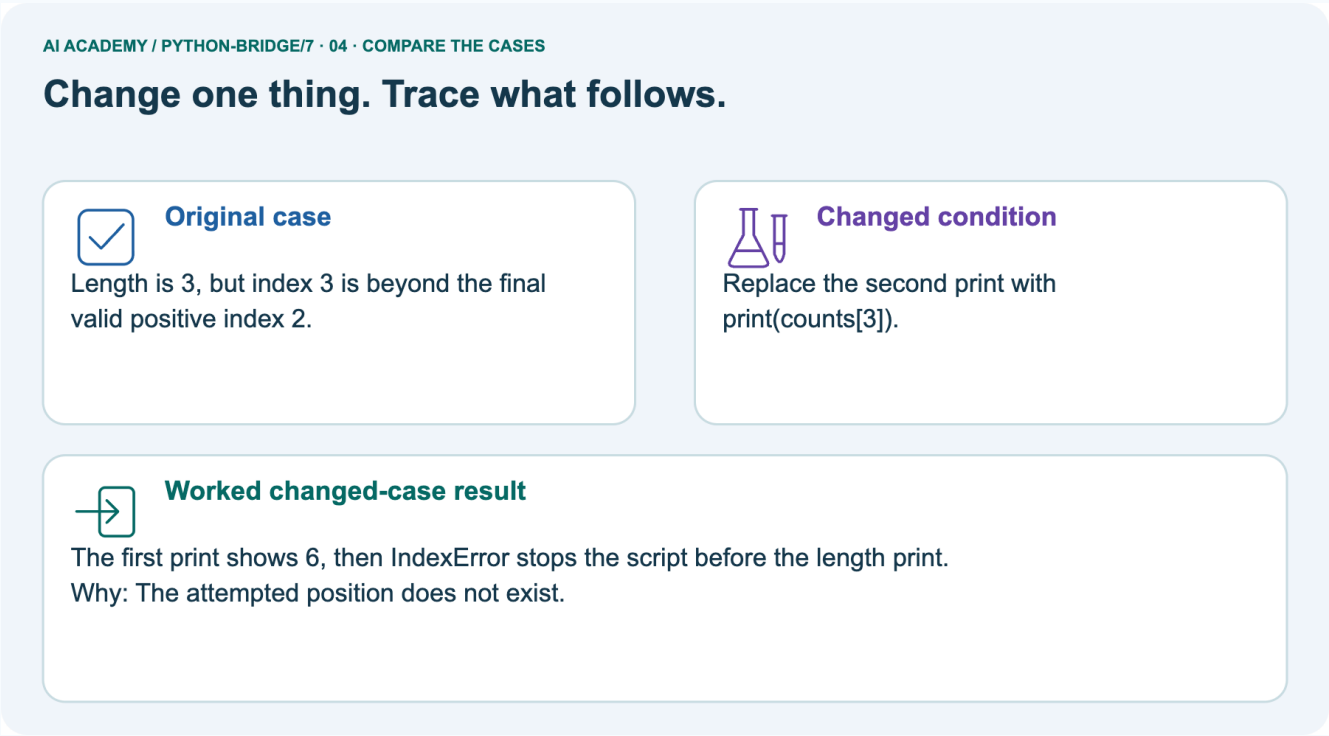
Read the labels and values. Explain the relationship the model shows.

#### Discuss while showing the picture

Teacher prompt: Read the labels and values. Explain the relationship the model shows. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Check row/column meanings, units and denominators before discussing a difference. Supplied toy values are not real class or model results.

## 04. Change one thing. Trace what follows.




Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2. Changed condition: Replace the second print with print(counts[3]).

Expected result: The first print shows 6, then **IndexError** stops the script before the **length** print. Reason: The attempted position does not exist.

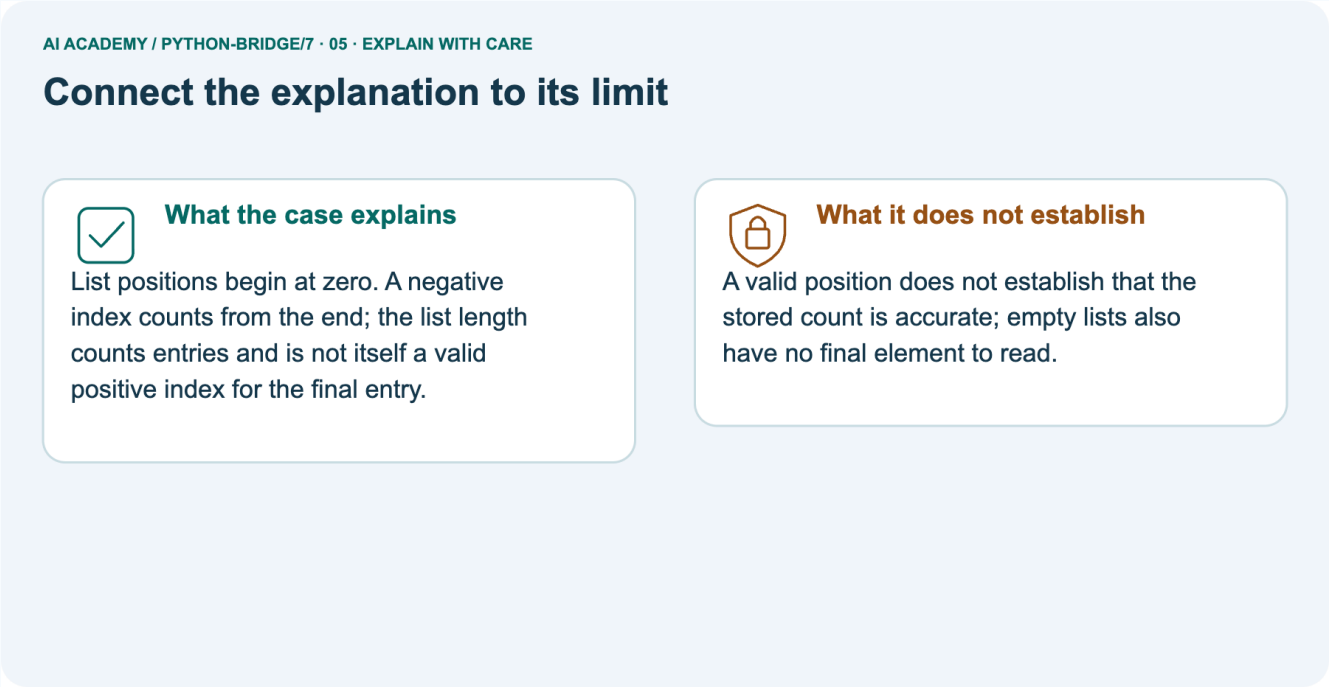
> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

### Discuss while showing the picture

Teacher prompt: Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude. Ask learners to describe one label or arrow in ordinary words.

Use the adjacent transcript if the image is difficult to see. Changed-case reasoning: The attempted position does not exist.

### 05. Connect the explanation to its limit




Follow the labelled picture with the matching explanation. Open the vector diagram

**List** positions begin at zero. A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

Limit: A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.

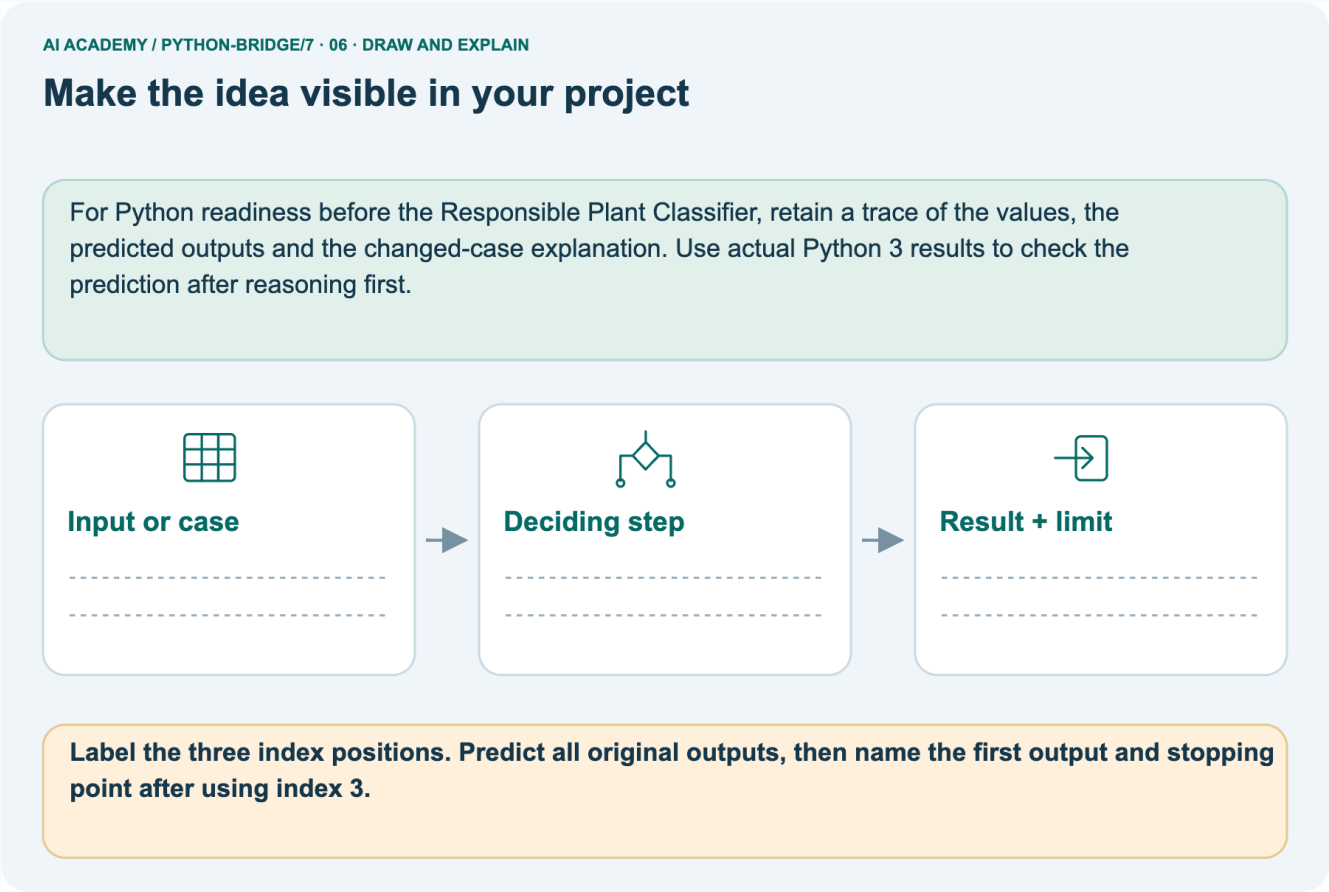
Explain the case, then point to a limit on the claim.

### Discuss while showing the picture

Teacher prompt: Explain the case, then point to a limit on the claim. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Keep the stated scope: A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.

## 06. Make the idea visible in your project




Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

Draw your project trace and justify the deciding step using **evidence**.

### Discuss while showing the picture

Teacher prompt: Draw your project trace and justify the deciding step using **evidence**. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Supported practice key: Original **output**s are 6,8,3. After the change, 6 prints first, counts[3] raises **IndexError**, and the third print does not execute.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.